REPORTE TÉCNICO: ENTREGA 1

RedSocialNFC · Arquitectura, Frameworks y Funcionamiento del Código

Materia: Desarrollo Móvil (Android)
Proyecto: RedSocialNFC
Rama GitHub: main (alaanvirtual)

Compilación: BUILD SUCCESSFUL (0 errores)

1. BIBLIOTECAS Y FRAMEWORKS UTILIZADOS

El proyecto fue construido utilizando el ecosistema oficial y moderno recomendado por Google para Android, asegurando un rendimiento nativo óptimo y compatibilidad declarativa:

Biblioteca / Framework	Versión	Propósito y Justificación Técnica
Jetpack Compose (BOM)	`2026.02.01`	Framework declarativo moderno de interfaz gráfica para Android. Elimina el uso de XML y permite construir interfaces reactivas basadas en funciones `@Composable` con re-composición inteligente.
Material Design 3 (Material3)	`1.3.x`	Sistema de diseño oficial de última generación. Provee componentes avanzados con accesibilidad nativa como `OutlinedTextField`, `Card`, `Surface`, `Scaffold` y soporte completo para paleta oscura dinámica.
AndroidX Room Database	`2.8.4`	Capa de abstracción sobre SQLite local. Provee mapeo objeto-relacional (ORM), verificación de consultas SQL en tiempo de compilación y persistencia segura de usuarios sin requerir conexión a internet.
KSP (Kotlin Symbol Processing)	`2.2.10-2.0.2`	Herramienta de procesamiento de anotaciones de alto rendimiento para Kotlin. Genera en tiempo de compilación las implementaciones concretas de `@Dao` y `@Database` de Room.
Kotlin Coroutines & Flow	`1.8.x`	Manejo de programación asíncrona no bloqueante. Ejecuta operaciones pesadas de base de datos en hilos secundarios mediante `suspend fun` y `rememberCoroutineScope()`.
SharedPreferences	Nativo Android	Almacenamiento clave-valor ligero en memoria para persistir la sesión abierta y fijar el identificador UUID único del hardware entre reinicios de la aplicación.
Material Icons Extended	`1.7.x`	Catálogo ampliado de iconografía vectorial oficial (candados, ojos de visibilidad, antena NFC, avatares, puertas de salida).
Android SDK & Gradle Plugin	`AGP 9.3.2 / SDK 37`	Herramientas de empaquetado, dexing y compilación del APK nativo con soporte retrocompatible desde Android 7.0 (MinSdk 24).

2. FUNCIONAMIENTO DETALLADO DEL CÓDIGO

El proyecto sigue el patrón arquitectónico limpio y modular visto a lo largo de las prácticas del curso, separando claramente la vista, el dominio, los datos y las utilidades:

A. Punto de Entrada y Navegación Reactiva (`MainActivity.kt`)

La aplicación utiliza una sola actividad (*Single Activity Architecture*) con renderizado de borde a borde (`enableEdgeToEdge()`). En lugar de depender de navegadores complejos, gestiona el estado de sesión de forma reactiva con Compose:

```
var sesionActiva by remember { mutableStateOf(sessionManager.haySesionActiva()) }
if (sesionActiva) {
    PerfilScreen(onCerrarSesion = { sesionActiva = false })
} else {
```

```
    AuthScreen(onLoginExitoso = { sesionActiva = true })
}
```

Al cambiar la variable `sesionActiva`, Compose recompone instantáneamente la interfaz cambiando entre la pantalla de autenticación y la pantalla de perfil sin destruir la actividad.

B. Capa de Datos y Persistencia SQLite (`Usuario.kt`, `UsuarioDao.kt`, `AppDatabase.kt`)

- **Entidad (`Usuario.kt`):** Modela la tabla `usuarios`. Almacena `id` (clave primaria autoincremental), `email`, `passwordHash`, `realHandle` (identidad real de 1er grado), `anonHandle` (identidad anónima de 2do grado) y `nfcUuid` (identificador criptográfico para la antena).
- **DAO (`UsuarioDao.kt`):** Interfaz que define las operaciones de acceso a datos mediante corrutinas:
  - `registrarUsuario(usuario: Usuario): Long`: Inserta el nuevo registro abortando duplicados.
  - `obtenerPorEmail(email: String): Usuario?`: Consulta parametrizada para validar login sin riesgo de inyección SQL.
  - `obtenerPorHandle(realHandle: String): Usuario?`: Verifica la unicidad del nombre de usuario.
- **Singleton Database (`AppDatabase.kt`):** Instancia única de la base de datos Room protegida por bloqueo sincronizado (`@Volatile` y `synchronized`) para evitar múltiples conexiones concurrentes.

C. Capa de Sesión y Criptografía de Hardware (`SessionManager.kt`)

Gestiona la persistencia liviana y contiene los algoritmos requeridos por la especificación:

- **RF-04 (Identificador Criptográfico NFC de Fondo):** La función `obtenerOGenerarDeviceNfcUuid()` consulta si el dispositivo ya tiene un UUID asignado en `SharedPreferences`. Si no existe, invoca `java.util.UUID.randomUUID().toString()`, generando un hash de 128 bits único y permanente para el hardware. Este proceso es completamente transparente para el usuario.
- **RF-03 (Generador de Identidad Anónima Inalterable):** La función `generarIdentidadAnonima()` combina números aleatorios entre 10 y 99 con caracteres alfabéticos (A-Z) para producir etiquetas del tipo `@Anonimo_89B`, garantizando anonimato en interacciones de 2do grado.

D. Pantalla de Autenticación y Registro (`AuthScreen.kt`)

Implementa la lógica interactiva de entrada con prevención preventiva de errores (UX):

- **RF-01 (Conmutador de Visibilidad de Contraseña):** Utiliza una variable booleana `passwordVisible`. Cuando es falsa, el campo activa `PasswordVisualTransformation()`; cuando es verdadera, usa `VisualTransformation.None`. El botón `IconButton` en el `trailingIcon` permite al usuario verificar lo escrito y corregir errores antes de enviar el formulario.
- **RF-02 (Handle Real con Prefijo Fijo):** Se implementa con `prefix = { Text("@") }` dentro de `OutlinedTextField`. Además, el evento `onValueChange` filtra activamente cualquier espacio o carácter no alfanumérico en tiempo real mediante `it.filter { it.isLetterOrDigit() || it == '_' }`.
- **RF-03 (Presentación de Identidad Inalterable):** El seudónimo anónimo autogenerado se renderiza dentro de un contenedor `Surface` estático decorado con una insignia **"FIJO"**. Al no utilizar un componente `TextField`, se imposibilita cualquier modificación o manipulación del texto por parte del usuario.
- **Validación Asíncrona:** Al pulsar "REGISTRAR IDENTIDAD", se lanza una corrutina (`scope.launch`) que verifica en la base de datos Room que el correo y el handle no se encuentren duplicados antes de confirmar la inserción.

E. Pantalla de Perfil de Identidad (`PerfilScreen.kt`)

Representa el panel de estado del usuario tras autenticarse:

- **Tarjeta de 1er Grado (Magenta Neón):** Destaca el `realHandle` con el color distintivo de contactos directos.
- **Tarjeta de 2do Grado (Gris-Violeta):** Presenta el `anonHandle` informando al usuario cómo lo visualizarán los contactos no vinculados.
- **Tarjeta de Hardware:** Expone el `nfcUuid` generado por el sistema que se usará para el sensor en la Entrega Final.
- **Cierre de Sesión Seguro:** Al presionar el botón de salida, se invoca `sessionManager.cerrarSesion()`, limpiando la memoria activa y devolviendo al usuario a la pantalla inicial.

3. ESPECIFICACIÓN DEL TEMA VISUAL (DIGITAL ALCHEMY)

El archivo `Color.kt` y `Theme.kt` definen el sistema cromático oscuro de alto contraste:

- ■ **DarkBackground (`#0B0813`):** Fondo inmersivo que reduce fatiga visual y consumo energético en pantallas OLED.
- ■ **NeonMagenta (`#FF007F`):** Color primario de acento para la identidad real y botones principales de acción.
- ■ **ElectricViolet (`#8A2BE2`):** Color secundario asignado a la capa de anonimato y vistas de 2do grado.
- ■ **NeonCyan (`#00E5FF`):** Acento terciario para indicadores criptográficos y hardware NFC.

4. RESUMEN DE REQUERIMIENTOS CUMPLIDOS

ID	Requerimiento Funcional	Control MD3 / Código	Resultado en Entrega 1
RF-01	Autenticación (Email / Password)	`OutlinedTextField` + `TrailingIcon`	Inicio y registro con conmutador de visibilidad funcional.
RF-02	Handle Real (@handle)	`OutlinedTextField` con `prefixText`	Formato forzado con prefijo '@' y filtrado alfanumérico.
RF-03	Identidad Anónima inalterable	`Surface / Text` estático	Generación automática (ej. @Anonimo_XX) de solo lectura.
RF-04	Identificador Criptográfico NFC	Proceso de fondo (`UUID`)	Asignación automática y persistencia en Room y SharedPreferences.